

Dedication

To my dear parents,

*for their unconditional love, sacrifices, prayers, and
constant support.*

To my family,

*for their encouragement, patience, and confidence
throughout this journey.*

To my teachers and mentors,

for their guidance, inspiration, and valuable advice.

I dedicate this work with deep gratitude and respect.

AIT-LAHCEN REDOUAN

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor for the guidance, availability, constructive feedback, and encouragement provided throughout this final year project. The advice received helped me clarify the project scope, strengthen the technical choices, and maintain a rigorous engineering approach.

I would also like to thank the academic staff for the knowledge and methodological foundations shared during my Master's program. Their teaching contributed directly to my understanding of data engineering, distributed systems, databases, software development, and reliable information system design.

My appreciation goes to the professional environment that supported the context of this project and helped me understand practical data engineering requirements. Working under the broader AdOptimizer AI initiative allowed me to position this work as a distributed customer data lakehouse platform focused on ingestion, distributed storage, lakehouse organization, transformation, quality validation, orchestration, observability, and security hardening.

I am grateful to my colleagues and classmates for their discussions, shared ideas, and support during this academic journey. I also extend my deepest thanks to my family for their patience, encouragement, and confidence during the long periods of research, implementation, testing, and documentation.

Finally, I would like to thank the members of the jury for the time and attention devoted to evaluating this work. I hope this report clearly reflects the effort invested in designing and implementing a modern, distributed, secure, and production-oriented data engineering platform.

Abstract

Modern organizations increasingly rely on data produced from multiple heterogeneous sources, including transactional systems, e-commerce platforms, behavioral event streams, and customer profile datasets. However, transforming these raw inputs into reliable, governed, and analytics-ready data products requires a structured data engineering platform capable of ingestion, storage, distributed processing, validation, orchestration, and monitoring.

This final year project presents the design and implementation of AdOptimizer CDP, a distributed customer data lakehouse platform developed under the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES. The platform focuses on building a scalable data engineering foundation for multi-source customer data. It integrates Apache Kafka for ingestion, Hadoop HDFS for distributed storage, Apache Spark for distributed processing, Apache Iceberg and Hive Metastore for lakehouse table management, Trino for SQL exploration, dbt-spark for transformation modeling, Great Expectations for data quality validation, Apache Airflow for orchestration, and Prometheus with Grafana for observability.

The project demonstrates an end-to-end architecture that transforms raw customer-related datasets into structured lakehouse layers and trusted analytical data products. The implemented solution emphasizes modularity, reproducibility, scalability, data quality, observability, security, and readiness for future analytics and machine learning use cases.

Keywords : Big Data, Data Engineering, Distributed Systems, Data Lakehouse, Apache Kafka, HDFS, Apache Spark, Apache Iceberg, Trino, dbt-spark, Data Quality, Observability, Security.

Résumé

Les organisations modernes exploitent des données issues de sources hétérogènes telles que les systèmes transactionnels, les plateformes e-commerce, les événements comportementaux et les données de profil client. Cependant, la transformation de ces données brutes en produits de données fiables, gouvernés et exploitables nécessite une plateforme d'ingénierie de données structurée, capable d'assurer l'ingestion, le stockage, le traitement distribué, la validation, l'orchestration et la supervision.

Ce projet de fin d'études présente la conception et la mise en oeuvre de AdOptimizer CDP, une plateforme lakehouse de données client distribuée réalisée dans le cadre de l'initiative plus large AdOptimizer AI chez SMART AUTOMATION TECHNOLOGIES. Le projet se concentre sur la construction d'une fondation d'ingénierie de données scalable pour des données client multi-sources. L'architecture intègre Apache Kafka pour l'ingestion, Hadoop HDFS pour le stockage distribué, Apache Spark pour le traitement distribué, Apache Iceberg et Hive Metastore pour la gestion des tables lakehouse, Trino pour l'exploration SQL, dbt-spark pour les transformations, Great Expectations pour la qualité des données, Apache Airflow pour l'orchestration, ainsi que Prometheus et Grafana pour l'observabilité.

La solution réalisée illustre une chaîne de traitement complète permettant de convertir des données client brutes en couches lakehouse structurées et en produits de données analytiques fiables. Elle met l'accent sur la modularité, la reproductibilité, la scalabilité, la qualité des données, l'observabilité, la sécurité et la préparation de futurs cas d'usage analytiques et machine learning.

Mots-clés : Big Data, Ingénierie des données, Systèmes distribués, Data Lakehouse, Apache Kafka, HDFS, Apache Spark, Apache Iceberg, Trino, dbt-spark, Qualité des données, Observabilité, Sécurité.

Contents

Dedication	i
Acknowledgements	ii
Abstract	iii
Résumé	iv
List of Acronyms and Abbreviations	xii
1 General Project Context	1
1.1 Introduction	1
1.2 Company and Strategic Context	1
1.3 Company Values and Their Relevance to the Project	2
1.4 Position of the Project Within AdOptimizer AI	3
1.5 Problem Statement	3
1.6 Motivation for a Distributed Lakehouse Approach	4
1.7 Project Positioning	4
1.8 Main Objectives of the Project	5
1.9 Implemented Scope and Project Boundaries	5
1.10 Expected Results and Academic Contribution	6
1.11 Conclusion	6
2 State of the Art and Theoretical Background	7
2.1 Introduction	7

2.2	Foundations of Modern Distributed Data Systems	7
2.2.1	From Centralized Systems to Distributed Data Platforms	7
2.2.2	Vertical Scaling vs Horizontal Scaling	8
2.2.3	Reliability, Scalability, and Maintainability	8
2.2.4	Data-Intensive Systems as a Distinct Engineering Problem	9
2.3	Core Theoretical Trade-offs in Distributed Data Engineering	9
2.3.1	The CAP Theorem and Consistency Trade-offs	9
2.3.2	ACID vs BASE Models	10
2.3.3	Data Partitioning and Replication	10
2.3.4	Fault Tolerance and Service Resilience	10
2.3.5	Batch Processing vs Stream Processing	11
2.3.6	Schema-on-Write vs Schema-on-Read	11
2.4	Big Data Foundations	11
2.4.1	The Five Vs of Big Data	11
2.4.2	Heterogeneous Data Sources and Data Variety	12
2.4.3	Distributed Storage Principles	12
2.4.4	Distributed Compute Principles	13
2.5	Evolution of Analytical Data Architectures	13
2.5.1	Traditional Data Warehouse Architecture	13
2.5.2	Data Lake Architecture	13
2.5.3	Data Lakehouse Architecture	14
2.5.4	Lambda Architecture	14
2.5.5	Kappa Architecture	14
2.5.6	Comparative Analysis of These Architectures	14
2.6	Foundational Concepts for Governance and Trust in Data Platforms	15
2.6.1	Metadata, Cataloging, and Data Discovery	15
2.6.2	Data Lineage and Traceability	15
2.6.3	Data Quality Dimensions	16
2.6.4	Observability in Distributed Data Systems	16

2.7	Security Foundations for Distributed Data Platforms	16
2.7.1	Authentication in Distributed Systems	16
2.7.2	Authorization and Access Control	17
2.7.3	Encryption in Transit and Encryption at Rest	17
2.7.4	Secure Service Exposure in Multi-Node Environments	17
2.8	Positioning of the Present Work	18
2.8.1	Theoretical Alignment with Distributed Lakehouse Principles	18
2.8.2	Why This Project Is a Data Engineering Platform and Not a Final AI Product	18
2.8.3	Positioning of AdOptimizer CDP within AdOptimizer AI	18
2.9	Conclusion	19
3	Technology Watch	20
3.1	Introduction	20
3.2	Selection Criteria	20
3.3	Ingestion and Messaging Technologies	20
3.4	Storage and Lakehouse Technologies	20
3.5	Processing, Query, and Transformation Technologies	21
3.6	Quality, Orchestration, and Monitoring Technologies	21
3.7	Security Technology Choices	21
3.8	Chapter Conclusion	21
4	Requirements Analysis and System Design	22
4.1	Introduction	22
4.2	Functional Requirements	22
4.3	Non-Functional Requirements	22
4.4	Actors and Usage Perspective	22
4.5	Global Logical Architecture	23
4.6	Physical Multi-Node Architecture	23
4.7	Design Rationale	23
4.8	Chapter Conclusion	23

5	Implementation of the Distributed Lakehouse Platform	24
5.1	Introduction	24
5.2	Data Sources and Dataset Modeling	24
5.3	Kafka-Based Ingestion Layer	24
5.4	HDFS Bronze Storage Layer	24
5.5	Spark Processing and Iceberg Loading	25
5.6	Hive Metastore and Catalog Governance	25
5.7	Trino Query Layer	25
5.8	dbt-spark Transformation Layer	25
5.9	Great Expectations Validation Layer	25
5.10	Airflow Orchestration Layer	25
5.11	Chapter Conclusion	25
6	Deployment Strategy and Distributed Execution	26
6.1	Introduction	26
6.2	Control Plane Responsibilities	26
6.3	VM-Level Service Separation	26
6.4	Data Flow Across Machines	26
6.5	Distributed Processing Proof	27
6.6	Operational Packaging and Deployment Bundles	27
6.7	Chapter Conclusion	27
7	Security Architecture and Hardening	28
7.1	Introduction	28
7.2	Security Objectives	28
7.3	Kerberos-Secured Hadoop Services	28
7.4	SPNEGO-Protected HDFS Browser Access	28
7.5	TLS-Protected Interfaces	29
7.6	Authenticated Trino Access	29
7.7	SSH-Based Remote Spark Submission	29

7.8	Encrypted Runtime Storage Support	29
7.9	Security Boundaries and Remaining Limits	29
7.10	Chapter Conclusion	29
8	Validation, Testing, and Results	30
8.1	Introduction	30
8.2	Validation Strategy	30
8.3	Infrastructure Validation	30
8.4	Pipeline Validation	30
8.5	Distributed Execution Validation	31
8.6	Security Validation	31
8.7	Observability and Monitoring Validation	31
8.8	Main Results	31
8.9	Chapter Conclusion	31
9	Discussion and Critical Analysis	32
9.1	Introduction	32
9.2	Strengths of the Final Architecture	32
9.3	Trade-Offs and Constraints	32
9.4	Academic Value	32
9.5	Industrial Value	33
9.6	Critical Limits	33
9.7	Chapter Conclusion	33
10	General Conclusion and Perspectives	34
10.1	General Conclusion	34
10.2	Main Achievements	34
10.3	Future Perspectives	34
10.4	Final Positioning	35
A	Appendices	37

List of Figures

1.1	SMART AUTOMATION TECHNOLOGIES visual identity	2
1.2	Core values of SMART AUTOMATION TECHNOLOGIES	2

List of Tables

List of Acronyms and Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CDP	Customer Data Platform
DAG	Directed Acyclic Graph
dbt	Data Build Tool
ETL	Extract, Transform, Load
GE	Great Expectations
HDFS	Hadoop Distributed File System
JDBC	Java Database Connectivity
JSONL	JSON Lines
KDC	Key Distribution Center
KPI	Key Performance Indicator
LTV	Lifetime Value
ML	Machine Learning
PFE	Projet de Fin d'Etudes
RBAC	Role-Based Access Control
SQL	Structured Query Language
SPNEGO	Simple and Protected GSSAPI Negotiation Mechanism
TLS	Transport Layer Security
UI	User Interface
VM	Virtual Machine

Chapter 1

General Project Context

1.1 Introduction

This chapter introduces the academic, business, and engineering context of AdOptimizer CDP. Its objective is to explain the environment in which the project was conducted, the broader strategic initiative that motivates it, the business problem it addresses, and the exact positioning of the present final year project within that larger transformation effort. It also clarifies the institutional origin of the work: the project was proposed and framed by SMART AUTOMATION TECHNOLOGIES as part of the professional context in which the present PFE was carried out.

1.2 Company and Strategic Context

AdOptimizer CDP was developed in the context of SMART AUTOMATION TECHNOLOGIES, the company that assigned and supported the project as the professional framework of this final year work. SMART AUTOMATION TECHNOLOGIES is a technology-oriented environment whose positioning is centered on digital innovation, automation, and intelligent systems. In the present project, the company context is important because it provides the industrial logic behind the need for a robust customer-data engineering platform. Rather than approaching data as a secondary reporting artifact, the project treats data as a strategic technical asset that must be ingested, governed, processed, exposed, validated, monitored, and secured in a coherent way.

The broader strategic direction in which this work is positioned is AdOptimizer AI. This global initiative represents the company's long-term vision for an intelligent advertising and marketing ecosystem able to support advanced customer understanding, future analytics, and data-driven optimization. The present PFE project was therefore not chosen in



Figure 1.1: SMART AUTOMATION TECHNOLOGIES visual identity

isolation; it was given by SMART AUTOMATION TECHNOLOGIES within that broader strategic orientation. However, such a vision cannot be implemented reliably without a strong data foundation. For that reason, the PFE contribution focuses specifically on the distributed customer-data platform layer required to support future intelligent services.

1.3 Company Values and Their Relevance to the Project

The company's institutional values provide a meaningful lens for understanding the spirit of the project. The visual material associated with SMART AUTOMATION TECHNOLOGIES highlights five major values: **Innovation**, **Team Spirit**, **Integrity**, **Excellence**, and **Commitment**. These values are not merely symbolic; they align naturally with the logic of a modern data engineering platform.



Figure 1.2: Core values of SMART AUTOMATION TECHNOLOGIES

Innovation is reflected in the adoption of a modern lakehouse architecture instead of a conventional isolated reporting workflow. Team spirit is reflected in the platform's role as a shared technical foundation that can later serve engineers, analysts, and future AI-oriented stakeholders. Integrity is reflected in the emphasis on trusted data, validation, reproducibility, and security. Excellence is reflected in the engineering objective of building a coherent end-to-end platform rather than assembling disconnected tools. Commitment is reflected in the effort to stabilize the platform operationally and make it defensible both technically and academically.

1.4 Position of the Project Within AdOptimizer AI

Within the broader AdOptimizer AI vision, the present PFE project must be positioned as AdOptimizer CDP, that is, the **distributed customer data lakehouse platform** that acts as the technical and engineering foundation of the larger initiative. This distinction is essential for the academic defense of the work.

The broader initiative is concerned with future intelligent marketing and advertising capabilities. Such capabilities may later include customer segmentation, conversion analysis, lifetime value analysis, churn-related reasoning, personalization, campaign optimization, and recommendation-oriented decision support. Nevertheless, these downstream intelligent functions are not the main implemented contribution of the present work. The actual contribution of the PFE is the design and implementation of the distributed and secure data platform that makes such future services possible and trustworthy.

This positioning is important because it prevents conceptual confusion during the defense. The project should not be described as a finished machine learning product, nor as a simple business intelligence dashboard. It must instead be defended as a Data Engineering and Big Data platform that delivers the foundational infrastructure necessary to sustain future Customer 360 and AI-driven use cases.

1.5 Problem Statement

Customer-related data is naturally fragmented across different operational contexts. In modern organizations, such fragmentation can appear through campaign-response datasets, digital behavioral traces, session-level intent indicators, and commercial transaction histories. When these data sources remain isolated, they weaken analytical trust, complicate reprocessing, reduce reproducibility, and make it difficult to construct governed datasets for downstream consumers.

From an engineering perspective, the problem is therefore not only to store data, but to create a structured platform able to ingest heterogeneous sources, preserve replayable raw artifacts, process them into governed lakehouse tables, expose them to analytical users, validate their quality, orchestrate execution reliably, and monitor operational behavior across a distributed environment. Without such a platform, future customer analytics remain fragile, difficult to scale, and hard to defend from a governance perspective.

The present project answers the following engineering question:

How can we build a distributed customer-data platform that ingests heterogeneous datasets through a reproducible pipeline, stores replayable raw data in distributed storage, processes it into governed lakehouse tables, exposes it for analytics, validates its quality, orchestrates it end to end, monitors the full system, and secures core infrastructure flows in a way that is defensible as a modern Big Data Engineering project?

1.6 Motivation for a Distributed Lakehouse Approach

The adoption of a distributed lakehouse architecture is motivated by both technical and academic reasons. From a technical point of view, the project needs to combine heterogeneous ingestion, replayable raw persistence, scalable processing, metadata governance, SQL exploration, transformation control, and data quality verification. From an academic point of view, the project must demonstrate that these capabilities are not isolated tasks, but parts of a coherent architectural response to a real-world data engineering problem.

The distributed lakehouse model is particularly appropriate because it allows the platform to keep raw data in distributed storage, process it with distributed compute resources, and expose curated analytical tables without forcing all data to be transformed immediately into a rigid warehouse-only model. It also supports separation of concerns between ingestion, storage, processing, transformation, validation, orchestration, and monitoring. This makes the architecture both more scalable and more defensible in front of a jury evaluating the project as a Data Engineering system rather than as a set of scripts.

1.7 Project Positioning

The final implemented platform is best described as a **secure distributed customer-data engineering platform**. Its role is to establish the technical conditions under which customer-related data can be treated as a governed and reusable organizational asset. In this sense, the project integrates ingestion, distributed storage, distributed processing, lakehouse

management, analytical querying, transformation governance, validation, orchestration, monitoring, and baseline infrastructure security into one operational foundation.

This positioning creates a strong academic identity for the report. The project is simultaneously:

- a Data Engineering platform,
- a Big Data architecture,
- a distributed lakehouse implementation,
- a secure multi-node data platform,
- and a deployment-oriented technical foundation for downstream Customer 360 and AI use cases.

1.8 Main Objectives of the Project

The project pursues a set of complementary objectives. First, it aims to organize heterogeneous customer-oriented datasets in a reproducible way. Second, it aims to transport these datasets through Kafka-based ingestion flows in order to create explicit and replayable movement patterns. Third, it aims to persist raw data in an HDFS bronze layer before converting it into structured analytical tables through Spark and Iceberg.

Beyond this core movement, the project also aims to centralize metadata through Hive Metastore, expose trusted analytical access through Trino, manage transformations through dbt-spark, validate data quality through Great Expectations, orchestrate the full sequence through Airflow, and monitor both the infrastructure and the pipelines through Prometheus, Grafana, and pipeline metrics. Finally, the project aims to demonstrate that these services can be deployed in a distributed and secured environment across multiple virtual machines rather than on a single isolated node.

1.9 Implemented Scope and Project Boundaries

The current implemented scope focuses on a customer-oriented multi-source bundle for Client 1. The platform integrates three active datasets that reflect complementary dimensions of customer understanding: campaign and profile data, online behavioral intent data, and commercial retail transaction data. This selection is sufficiently rich to justify future Customer 360 analytics while remaining compatible with the available infrastructure.

At the same time, it is important to state the project’s boundaries clearly. The present work does not implement the final downstream machine learning modules such as dynamic segmentation, churn prediction, lifetime value prediction, persona generation, or recommendation engines. Instead, it implements the distributed and secure data foundation required for such future capabilities. This distinction strengthens the rigor of the report because it prevents exaggerated claims and keeps the academic focus on the actual engineering contribution.

1.10 Expected Results and Academic Contribution

The expected outcome of the project is a reproducible end-to-end data platform in which heterogeneous customer-related datasets can be ingested, stored, processed, transformed, queried, validated, orchestrated, monitored, and secured. Concretely, the platform should produce replayable bronze persistence in distributed storage, trusted analytical tables available through SQL, validation evidence usable in the final defense, and a distributed runtime architecture that demonstrates collaboration across multiple nodes.

From an academic perspective, the project contributes by showing how a modern customer-data platform can be designed as a coherent integration of event transport, distributed storage, distributed computation, metadata governance, analytical access, transformation control, quality assurance, orchestration, observability, and security hardening. The originality of the work does not lie in inventing new algorithms, but in engineering a defensible distributed platform architecture that connects these concerns into one consistent implementation.

1.11 Conclusion

This chapter established the industrial, business, and academic framing of AdOptimizer CDP. It clarified the role of SMART AUTOMATION TECHNOLOGIES, the positioning of the project within AdOptimizer AI, the nature of the underlying business problem, the motivation for a distributed lakehouse solution, the implemented objectives, and the limits of the present contribution. On this basis, the next chapter can now move from contextual framing toward the theoretical foundations that justify the architectural and engineering choices made in the project.

Chapter 2

State of the Art and Theoretical Background

2.1 Introduction

This chapter establishes the academic and theoretical foundation required to defend AdOptimizer CDP rigorously. The implemented platform is not only the result of practical integration between several tools; it is the consequence of architectural choices made under well-known constraints in distributed data engineering. For that reason, the chapter begins with global concepts and design trade-offs before moving toward architectural models. Its objective is to explain why a distributed customer-data platform must be designed in layers, why storage and compute cannot be treated as isolated concerns, and why governance, observability, and security are not optional additions but structural requirements.

The discussion is informed by the broader literature on distributed and data-intensive systems, including the design logic emphasized by Martin Kleppmann in *Designing Data-Intensive Applications*, especially around reliability, scalability, maintainability, partitioning, replication, and consistency trade-offs. The goal is not to summarize the entire field, but to identify the concepts that directly justify the architecture adopted later in this report.

2.2 Foundations of Modern Distributed Data Systems

2.2.1 From Centralized Systems to Distributed Data Platforms

Traditional information systems were largely organized around centralized databases and a relatively small number of application servers. This model remains effective when data volume is moderate, operational contexts are stable, and the number of consumers remains

limited. However, once organizations begin collecting heterogeneous data from multiple channels, centralized architectures become difficult to sustain. Storage pressure increases, computational workloads become more varied, and the system becomes more sensitive to single-node bottlenecks.

Distributed data platforms emerged as a response to these limitations. Their purpose is not only to increase raw capacity, but also to separate responsibilities according to operational logic. In a modern platform, event transport, raw storage, distributed computation, metadata management, analytical exposure, data quality validation, and observability can be handled by different services while still forming a coherent whole. This decomposition allows each concern to evolve with the technical model best suited to it.

2.2.2 Vertical Scaling vs Horizontal Scaling

One of the first decisions in large-scale systems concerns how growth should be absorbed. Vertical scaling strengthens a machine by increasing CPU, memory, or storage capacity. It is often simpler in the short term because the system does not need to be partitioned immediately. Nevertheless, it has two major limits: physical growth is finite, and dependence on a small number of powerful machines weakens resilience.

Horizontal scaling increases capacity by adding nodes. This model is more complex because it requires workload distribution, coordination, and partial-failure management. However, it offers better long-term elasticity and better alignment with modern data engineering workloads. Storage can be distributed across nodes, compute can run in parallel, and replication strategies can reduce the risk of complete service interruption. For a distributed customer-data platform, horizontal scaling is therefore not just a performance technique; it is a structural architectural principle.

2.2.3 Reliability, Scalability, and Maintainability

The literature on distributed systems frequently emphasizes three central qualities: reliability, scalability, and maintainability. Reliability refers to the system's ability to continue providing correct service despite component failures, abnormal workloads, or operational disturbances. Scalability refers to the platform's capacity to grow without disproportionate degradation or redesign. Maintainability refers to the ability to understand, operate, extend, and debug the system over time.

These three qualities are fundamental for a platform such as AdOptimizer CDP. A customer-data foundation that is unreliable cannot support trusted analytical reuse. A platform that is not scalable cannot sustain new datasets, new clients, or future machine learning use cases. A platform that is not maintainable becomes fragile and difficult to defend

academically because its behavior depends on hidden assumptions and manual intervention. Consequently, these qualities are not abstract ideals; they are direct evaluation criteria for the engineering relevance of the project.

2.2.4 Data-Intensive Systems as a Distinct Engineering Problem

Data-intensive systems differ from ordinary software systems because data movement, storage, replayability, and consistency become first-class engineering concerns. In such systems, correctness is not defined only by the logic of business functions. It also depends on whether data is durably persisted, whether transformations are reproducible, whether schemas remain interpretable over time, and whether the system can explain the origin and lifecycle of the information it exposes.

This distinction is important for the academic positioning of the present work. AdOptimizer CDP is not simply an application with a database behind it. It is a distributed data engineering platform whose value lies in organizing the lifecycle of heterogeneous customer-related data in a reliable, observable, and governable way. Therefore, the project must be understood through the conceptual framework of data-intensive and distributed systems.

2.3 Core Theoretical Trade-offs in Distributed Data Engineering

2.3.1 The CAP Theorem and Consistency Trade-offs

The CAP theorem remains one of the best-known conceptual references in distributed systems. It states that under network partition conditions, a distributed system cannot simultaneously provide both strong consistency and full availability. Although this statement is often simplified in practice, its educational importance is clear: once a system is distributed, architects must explicitly decide how consistency, availability, and failure tolerance are balanced.

For data platforms, this means that different layers of the architecture can make different trade-offs. Event transport systems, distributed filesystems, metadata services, and analytical table layers are not expected to behave identically. Some prioritize durability and replayability, others prioritize read scalability, and others introduce stronger transactional semantics for curated analytical datasets. The value of the theorem in the present context is therefore not to impose one universal answer, but to justify why a multi-layer architecture is necessary.

2.3.2 ACID vs BASE Models

ACID properties traditionally define the transactional expectations of strongly controlled database systems: atomicity, consistency, isolation, and durability. These properties are essential when every update must remain immediately and strictly correct. However, not every stage of a distributed data platform requires the same guarantees at the same time.

BASE, commonly summarized as Basically Available, Soft state, Eventual consistency, represents a different philosophy. It accepts temporary divergence and asynchronous convergence when this allows better distribution, availability, or throughput. Many ingestion and transport layers operate closer to BASE-like behavior, whereas curated analytical layers often benefit from stronger ACID-like semantics. This explains the importance of table formats and catalog layers that reintroduce stronger transactional guarantees on top of flexible distributed storage.

2.3.3 Data Partitioning and Replication

Partitioning divides data into smaller units so that storage and computation can be distributed across multiple nodes. It is one of the foundational mechanisms behind modern large-scale platforms. Without partitioning, a single node becomes a natural bottleneck for read throughput, write throughput, fault recovery, and transformation execution.

Replication complements partitioning by improving resilience and availability. When multiple copies of data or logs exist, the system can continue operating even when one node is degraded or unavailable. Yet replication is not free: it introduces coordination overhead and consistency challenges. This is why partitioning and replication should not be viewed as purely implementation-level details. They are fundamental architectural instruments for balancing scale, resilience, and operational complexity.

2.3.4 Fault Tolerance and Service Resilience

In distributed systems, failure must be considered normal rather than exceptional. Nodes may disappear, ports may become temporarily unavailable, certificates may expire, services may start in the wrong order, and storage volumes may create cascading readiness issues. A resilient platform therefore needs explicit strategies for retries, health checks, replication, recovery, and replay.

This principle is particularly relevant in data engineering because failures can affect not only uptime but also data trust. A silent ingestion failure may result in missing data. A metadata failure may make tables unreadable. A readiness misconfiguration may cause an orchestration layer to fail even when the data itself is correct. Therefore, fault tolerance is

directly tied to platform credibility and reproducibility.

2.3.5 Batch Processing vs Stream Processing

Batch processing is based on finite datasets processed periodically or on demand. It is suitable for historical backfills, curated analytical transformations, and reproducible large-scale recomputation. Stream processing is based on continuous flows of records or events that are handled incrementally as they arrive. It is useful when ingestion or propagation must remain closer to real-time behavior.

These two paradigms should not be treated as mutually exclusive. In modern platforms, they often coexist. Event streaming can capture and transport raw information continuously, while downstream computation may structure the resulting data into trusted analytical tables through batch or micro-batch logic. This hybrid view is especially relevant in customer-data engineering, where event freshness matters but governance and curated analytical reuse remain essential.

2.3.6 Schema-on-Write vs Schema-on-Read

Schema-on-write means that data must be shaped according to a predefined structure before it is stored in an authoritative analytical repository. This approach improves control and downstream semantic stability, but it may reduce ingestion flexibility. Schema-on-read, by contrast, stores raw or semi-structured data first and interprets structure later when the data is processed or queried.

This trade-off is central to modern lakehouse thinking. Raw layers benefit from schema-on-read because they preserve source fidelity and support replay. Curated layers benefit from stronger schema discipline because users require stable and understandable analytical entities. A layered data platform therefore does not simply choose one strategy over the other. It combines them in sequence, using flexibility for raw persistence and stronger structure for curated consumption.

2.4 Big Data Foundations

2.4.1 The Five Vs of Big Data

Big Data is often introduced through five dimensions: volume, velocity, variety, veracity, and value. Volume refers to the quantity of data that must be managed. Velocity refers to the rate at which data is generated and moved. Variety refers to the diversity of formats,

structures, and source semantics. Veracity concerns the reliability and quality of the data. Value concerns the usefulness that can be extracted from it for decision-making and future intelligent services.

These dimensions remain relevant because they show that data engineering complexity does not depend only on scale. Even when data volume remains manageable, high variety, high reuse pressure, and high governance expectations can justify a distributed architecture. This is particularly true for customer-data platforms, where datasets often differ in structure, semantics, and analytical purpose.

2.4.2 Heterogeneous Data Sources and Data Variety

Customer understanding rarely comes from a single operational system. It may involve campaign response datasets, online behavioral signals, e-commerce or retail transactions, session-level intent indicators, and profile-related attributes. Each source differs in format, cadence, completeness, and downstream meaning. This heterogeneity makes integration difficult because technical normalization is not sufficient; semantic reconciliation is also required.

As a result, variety becomes both a storage problem and a modeling problem. The platform must be able to ingest different raw forms, persist them durably, and later organize them into analytical layers without erasing their provenance. This is one of the strongest arguments for using layered architecture rather than directly forcing all incoming data into one rigid warehouse model.

2.4.3 Distributed Storage Principles

Distributed storage spreads persisted data across multiple nodes instead of concentrating it on a single machine. Its goals include capacity growth, resilience, replication, parallel read/write support, and better alignment with distributed compute. An important consequence of distributed storage is that it separates durable raw persistence from the compute jobs that later transform the data.

This separation has strong academic value. When raw data is durably preserved, pipelines become replayable and easier to audit. Transformations can be rerun without repeatedly depending on the original upstream source. The raw layer therefore serves both operational recovery and methodological reproducibility. In a PFE context, this is a strong argument because it shows that architecture supports not only execution but also traceable engineering reasoning.

2.4.4 Distributed Compute Principles

Distributed compute allows processing tasks to be decomposed and executed across several nodes. Instead of one machine reading and transforming an entire dataset, work is split according to partitions, files, or execution stages. This can reduce total execution time and align compute more closely with distributed storage.

However, distributed compute is beneficial only when the workload justifies it. Task scheduling, serialization, dependency management, and cluster readiness all introduce overhead. Therefore, an academically defensible platform must show that distributed compute is not used only as a slogan. It must be used where the problem structure, data movement pattern, and storage organization make parallel execution meaningful.

2.5 Evolution of Analytical Data Architectures

2.5.1 Traditional Data Warehouse Architecture

The traditional data warehouse is organized around structured, curated, and strongly modeled analytical data. It is highly effective for stable reporting, dimensional modeling, and SQL-based decision support. Its main strength is semantic control.

Its limitation appears when heterogeneous or semi-structured data must be ingested rapidly and preserved before full normalization. In such contexts, the warehouse-only model can become too rigid because it expects analytical structure too early in the lifecycle. This rigidity is often incompatible with modern customer-data environments where raw preservation and later reinterpretation are valuable.

2.5.2 Data Lake Architecture

The data lake emerged as a response to the rigidity of traditional warehouses. It enables large quantities of raw and semi-structured data to be stored before full modeling. This increases ingestion flexibility and supports exploratory reuse across different downstream consumers.

However, data lakes also introduced a new risk: if metadata, naming discipline, and validation remain weak, the platform can degrade into an opaque storage zone with low analytical trust. In other words, the lake model solves flexibility problems but can create governance problems. This is why the data lake alone is rarely sufficient for a mature customer-data platform.

2.5.3 Data Lakehouse Architecture

The data lakehouse seeks to combine the flexibility of the lake with the transactional and governance discipline associated with warehouses. It preserves distributed and replayable storage logic while adding stronger table semantics, better schema evolution control, and improved analytical exposure.

This model is particularly well adapted to projects such as AdOptimizer CDP. It allows raw datasets to be preserved in distributed storage while curated datasets are exposed through governed tables suitable for transformation frameworks and SQL query engines. The lakehouse can therefore be defended as an architectural response to the need for both flexibility and trust.

2.5.4 Lambda Architecture

Lambda architecture organizes processing through two paths: a batch path for full historical recomputation and a speed layer for low-latency updates. Its objective is to reconcile correctness and freshness. The model was influential because it formalized the difficulty of satisfying both needs through a single pipeline.

Nevertheless, Lambda architecture is often criticized for operational duplication. Two processing logics must be maintained, monitored, and reconciled. Its academic relevance remains high because it helps explain why organizations later looked for simplified architectures able to keep event responsiveness without multiplying engineering burden.

2.5.5 Kappa Architecture

Kappa architecture reduces the conceptual duplication of Lambda by emphasizing a stream-centered approach in which the event log becomes the main source of truth. Reprocessing is achieved through replay rather than through a separate batch computation layer.

This model is attractive when event logs are rich, retention is strong, and the platform is comfortable treating stream replay as its main recovery and recomputation mechanism. However, many real analytical platforms still require clearly governed historical layers and structured table abstractions. Therefore, Kappa is instructive, but not automatically sufficient for every customer-data use case.

2.5.6 Comparative Analysis of These Architectures

The historical movement from warehouses to lakes, from lakes to lakehouses, and from isolated reporting toward event-aware architectures reflects a broader shift in priorities.

Early architectures privileged control and stable relational semantics. Later architectures privileged ingestion flexibility and scale. Contemporary platforms increasingly seek a hybrid answer that preserves raw fidelity, enables distributed processing, and still produces governed analytical products.

For AdOptimizer CDP, the lakehouse model is the most suitable. It supports raw bronze persistence, distributed storage, structured analytical tables, metadata-driven reuse, and downstream SQL exploration. It also remains compatible with event-driven ingestion patterns and security hardening requirements. As a result, the selected architecture is not arbitrary; it is a reasoned synthesis of the state of the art.

2.6 Foundational Concepts for Governance and Trust in Data Platforms

2.6.1 Metadata, Cataloging, and Data Discovery

Metadata gives meaning to stored data by describing structure, origin, semantics, and technical location. Without metadata, even accessible data can remain practically unusable. Cataloging extends this principle by making datasets discoverable and interpretable by different actors.

In a distributed customer-data platform, metadata is especially important because the architecture spans several services and layers. Governance cannot depend on informal memory alone. A defensible platform must therefore treat schema registration, table visibility, and dataset discovery as explicit capabilities.

2.6.2 Data Lineage and Traceability

Lineage refers to the ability to trace how data moved from source to output, which transformations were applied, and which downstream assets depend on which upstream artifacts. Traceability is essential for reproducibility, debugging, and auditability.

From an academic point of view, lineage helps distinguish a genuine data engineering platform from a simple collection of disconnected scripts. When lineage can be reconstructed, the platform becomes explainable. This is essential for a PFE project that must defend not only outcomes but also the logic by which those outcomes were produced.

2.6.3 Data Quality Dimensions

Data quality is multidimensional. Common dimensions include completeness, consistency, validity, accuracy, timeliness, and uniqueness. These dimensions are especially important in customer-data contexts because downstream segmentation, profiling, or future predictive analytics become weak when source data is inconsistent or poorly governed.

Modern platforms increasingly formalize quality as an explicit layer of the architecture rather than as a purely manual cleaning activity. Checks, expectations, thresholds, and validation artifacts transform quality into a monitored engineering property. This move is important because it makes trust operationally visible.

2.6.4 Observability in Distributed Data Systems

Observability is the capacity to infer the internal state of a system through emitted evidence such as metrics, logs, traces, health checks, and runtime indicators. In distributed environments, observability is crucial because failures often emerge from the interaction between components rather than from one isolated service.

For data engineering, observability is tightly linked to platform trust. If the system cannot show whether ingestion succeeded, whether storage is healthy, whether services are reachable, or whether transformations produced expected results, its outputs become difficult to defend. Observability is therefore part of the platform's academic credibility as well as its operational practicality.

2.7 Security Foundations for Distributed Data Platforms

2.7.1 Authentication in Distributed Systems

Authentication establishes who a user or service is. In distributed environments, it becomes critical because communication crosses process, host, and network boundaries. Storage services, metadata layers, and query endpoints must be able to distinguish legitimate principals from unauthorized ones.

Without strong authentication, a distributed platform depends on implicit trust between components, which becomes increasingly dangerous as the number of services grows. Therefore, authentication is a structural foundation of secure distributed data engineering.

2.7.2 Authorization and Access Control

Once identity is established, the system must determine which actions are permitted. Authorization controls access to services, datasets, administrative operations, and query interfaces. It operationalizes the principle of least privilege by restricting each actor to the minimum access necessary for its role.

In data platforms, authorization cannot be limited to end-user interfaces alone. It also applies to service-to-service calls, operational scripts, metadata access, and internal gateways. A mature distributed architecture must therefore combine authentication and authorization rather than treating them as separate afterthoughts.

2.7.3 Encryption in Transit and Encryption at Rest

Encryption in transit protects data while it moves between nodes, services, or users. It reduces exposure to interception and helps preserve confidentiality across the network. Encryption at rest protects persisted data on storage media and reduces exposure if storage devices or mounted runtime paths are accessed outside intended controls.

These two mechanisms answer different threat surfaces and must be treated as complementary. In a multi-service lakehouse platform, data frequently moves through ingestion, orchestration, query, and monitoring channels. Consequently, transport security and persistent storage protection both belong to the theoretical foundation of a defensible architecture.

2.7.4 Secure Service Exposure in Multi-Node Environments

Distributed platforms expose numerous service endpoints: orchestration UIs, query interfaces, storage consoles, monitoring dashboards, and service APIs. If these are exposed without proxying, credential checks, and transport protection, the system accumulates a wide attack surface.

Secure exposure therefore requires deliberate control of entry points, certificates, internal-versus-external interfaces, and service-access boundaries. This is particularly relevant to AdOptimizer CDP because one of its core challenges is not only to run multiple services, but to expose them in a way that remains manageable, authenticated, and defensible.

2.8 Positioning of the Present Work

2.8.1 Theoretical Alignment with Distributed Lakehouse Principles

The architecture of AdOptimizer CDP aligns with distributed lakehouse principles because it separates raw ingestion, distributed persistence, curated analytical structuring, metadata management, validation, orchestration, and observability into coordinated layers. This layered model directly reflects the theoretical distinctions discussed earlier in the chapter: flexible raw persistence, stronger structure for trusted curated layers, distributed compute for transformation, and governance for analytical reuse.

2.8.2 Why This Project Is a Data Engineering Platform and Not a Final AI Product

Although the broader AdOptimizer AI initiative may later incorporate advanced analytical and intelligent marketing functions, the present work should not be described as a completed AI product. Its implemented contribution lies in the design and deployment of the distributed data platform that makes future analytics possible under reproducible, governed, and secure conditions.

This distinction is academically essential. It prevents exaggerated claims and keeps the defense aligned with the real contribution of the project: distributed storage, distributed processing, analytical structuring, data quality control, orchestration, monitoring, and infrastructure hardening. These are substantial engineering results in their own right.

2.8.3 Positioning of AdOptimizer CDP within AdOptimizer AI

Within AdOptimizer AI, AdOptimizer CDP plays the role of the distributed customer-data foundation. It is the technical layer responsible for receiving heterogeneous inputs, organizing them into replayable and structured movement patterns, persisting them durably, curating them into analytical datasets, and exposing them to future consumers under controlled conditions.

This positioning gives the project a clear strategic identity. It is neither an isolated classroom exercise nor a finished business intelligence product. It is a concrete infrastructure contribution embedded in a larger company initiative while remaining academically defensible as a modern Big Data and Data Engineering platform.

2.9 Conclusion

This chapter established the theoretical and architectural background that justifies the design of AdOptimizer CDP. It introduced the evolution of distributed data systems, the principal engineering trade-offs that shape them, the foundations of Big Data platforms, the comparative logic of modern analytical architectures, and the core principles of governance, observability, and security in distributed environments.

On this basis, the following chapters can now move from theory to requirements, design, and implementation. The platform can therefore be presented not as a collection of tools, but as a coherent response to well-identified distributed data engineering challenges.

Chapter 3

Technology Watch

3.1 Introduction

This chapter explains the technology selection process used to assemble the final platform.

3.2 Selection Criteria

The comparison should consider compatibility with distributed deployment, maturity, documentation quality, ecosystem adoption, security capabilities, operational simplicity, and suitability for academic demonstration.

3.3 Ingestion and Messaging Technologies

This section should explain the selection of Kafka for event-driven ingestion and controlled topic-based data movement.

3.4 Storage and Lakehouse Technologies

This section should justify the use of HDFS for distributed storage and Iceberg for managed analytical table layers.

3.5 Processing, Query, and Transformation Technologies

This section should justify the use of Spark for distributed processing, Trino for SQL exposure, and dbt-spark for transformation governance.

3.6 Quality, Orchestration, and Monitoring Technologies

This section should justify Great Expectations, Airflow, Prometheus, Grafana, and supporting observability services.

3.7 Security Technology Choices

This section should justify Kerberos, SPNEGO, TLS, SSH-based submission, and encrypted storage support.

3.8 Chapter Conclusion

This chapter should show that the final stack was selected for coherence, defensibility, and practical integration rather than for tool accumulation.

Chapter 4

Requirements Analysis and System Design

4.1 Introduction

This chapter formalizes the functional and non-functional requirements of AdOptimizer CDP and translates them into an architectural design.

4.2 Functional Requirements

The platform must support heterogeneous data ingestion, distributed storage, distributed transformation, catalog integration, SQL access, orchestration, monitoring, and validation workflows.

4.3 Non-Functional Requirements

The platform must demonstrate scalability, modularity, security, observability, service separation, reproducibility, and suitability for academic demonstration.

4.4 Actors and Usage Perspective

This section should identify key actors such as data engineers, analysts, platform administrators, and future data science consumers.

4.5 Global Logical Architecture

This section should present the logical data flow from ingestion to validation and analytical exposure.

4.6 Physical Multi-Node Architecture

This section should present the role separation across the control plane, VM1, VM2, and VM3.

4.7 Design Rationale

The final design should be explained as a balance between academic rigor, distributed execution proof, security hardening, and resource constraints.

4.8 Chapter Conclusion

This chapter provides the design baseline for the concrete implementation detailed in the next chapter.

Chapter 5

Implementation of the Distributed Lakehouse Platform

5.1 Introduction

This chapter presents the concrete implementation of AdOptimizer CDP from ingestion to analytical serving.

5.2 Data Sources and Dataset Modeling

This section should describe the selected customer-oriented datasets, the reason for their retention, and their role in the platform.

5.3 Kafka-Based Ingestion Layer

This section should describe topic organization, producers, consumers, and ingestion orchestration logic.

5.4 HDFS Bronze Storage Layer

This section should describe bronze storage organization, batch landing patterns, and the role of distributed persistence.

5.5 Spark Processing and Iceberg Loading

This section should explain Spark jobs, schema handling, loading behavior, and the creation of analytical Iceberg tables.

5.6 Hive Metastore and Catalog Governance

This section should explain how metadata is centralized and exposed to upper analytical services.

5.7 Trino Query Layer

This section should explain SQL access patterns, query federation role, and secure access exposure.

5.8 dbt-spark Transformation Layer

This section should explain transformation modeling, model organization, and governed analytical layer creation.

5.9 Great Expectations Validation Layer

This section should explain quality checks and their role in proving governance and trustworthiness.

5.10 Airflow Orchestration Layer

This section should explain the DAG structure, sequencing, operational controls, and restart logic.

5.11 Chapter Conclusion

This chapter should demonstrate that the platform is implemented as a coherent system rather than as isolated components.

Chapter 6

Deployment Strategy and Distributed Execution

6.1 Introduction

This chapter explains how the platform is deployed physically and how distributed execution is demonstrated across multiple nodes.

6.2 Control Plane Responsibilities

This section should describe the local control plane services, their role, and why they remain separated from the data-plane virtual machines.

6.3 VM-Level Service Separation

This section should explain why VM1, VM2, and VM3 host different services and how this separation supports resilience, modularity, and clearer operational reasoning.

6.4 Data Flow Across Machines

The report should clearly explain the end-to-end movement of data:

- source datasets to Kafka,
- Kafka to HDFS bronze,

- HDFS to Spark,
- Spark to Iceberg,
- Iceberg and Metastore to Trino,
- transformations and validation to final analytical exposure.

6.5 Distributed Processing Proof

This section should explain how Spark workers collaborate across nodes and how the deployment proves multi-node execution rather than single-machine processing.

6.6 Operational Packaging and Deployment Bundles

This section should document the deployment bundles, environment preparation, startup scripts, and operational checks used to launch the platform.

6.7 Chapter Conclusion

This chapter should prove that the deployment is intentionally distributed, reproducible, and academically defensible.

Chapter 7

Security Architecture and Hardening

7.1 Introduction

This chapter explains the security controls introduced into AdOptimizer CDP and their role in protecting the distributed platform.

7.2 Security Objectives

The security layer aims to protect service access, secure browser-facing interfaces, control distributed identity, and reduce exposure of operational services.

7.3 Kerberos-Secured Hadoop Services

This section should explain the use of Kerberos for Hadoop authentication and why secure identity exchange is important in a distributed system.

7.4 SPNEGO-Protected HDFS Browser Access

This section should explain how browser access to HDFS was protected and why this matters for administrative interfaces.

7.5 TLS-Protected Interfaces

This section should explain the TLS layer added to exposed services and the distinction between internal service traffic and browser-facing endpoints.

7.6 Authenticated Trino Access

This section should explain Trino gateway protection, authenticated access, and the user-facing implications of secure SQL exposure.

7.7 SSH-Based Remote Spark Submission

This section should explain how remote execution from the control plane to VM2 is secured operationally.

7.8 Encrypted Runtime Storage Support

This section should explain the role of encrypted runtime storage and its contribution to protecting data at rest.

7.9 Security Boundaries and Remaining Limits

This section should present the implemented scope honestly and distinguish completed hardening from future enterprise-grade enhancements.

7.10 Chapter Conclusion

This chapter should show that security was integrated as an architectural concern rather than added as a superficial afterthought.

Chapter 8

Validation, Testing, and Results

8.1 Introduction

This chapter presents the validation methodology and the main technical results obtained from the deployed platform.

8.2 Validation Strategy

The validation should cover service readiness, distributed execution, ingestion success, storage correctness, analytical exposure, transformation success, quality controls, and observability.

8.3 Infrastructure Validation

This section should summarize the checks performed for Kafka, HDFS, Hive Metastore, Spark, Trino, Kerberos, TLS gateways, and monitoring services.

8.4 Pipeline Validation

This section should present the main DAG execution results and explain how the orchestration proves operational readiness.

8.5 Distributed Execution Validation

This section should provide evidence that multiple Spark workers and virtual machines collaborate during processing.

8.6 Security Validation

This section should present authentication and protected-access checks, especially for HDFS browser access and secured analytical services.

8.7 Observability and Monitoring Validation

This section should summarize the role of Grafana, Prometheus, cAdvisor, and pipeline metrics in validating operational visibility.

8.8 Main Results

This section should present the overall technical outcome: the platform is deployed, secure, distributed, orchestrated, and usable as a foundation for future higher-level analytics.

8.9 Chapter Conclusion

This chapter should demonstrate that the project is operationally validated rather than only theoretically designed.

Chapter 9

Discussion and Critical Analysis

9.1 Introduction

This chapter provides a critical interpretation of the implemented work.

9.2 Strengths of the Final Architecture

This section should analyze the main strengths of the project, including modularity, service separation, distributed execution, governance orientation, and security integration.

9.3 Trade-Offs and Constraints

This section should discuss practical constraints such as limited infrastructure resources, multi-service integration complexity, and the operational overhead of a secure distributed architecture.

9.4 Academic Value

This section should explain the academic contribution of the project in terms of data engineering, distributed systems, lakehouse design, orchestration, observability, and platform security.

9.5 Industrial Value

This section should explain how the platform creates a usable foundation for future customer intelligence and AI services inside the broader initiative.

9.6 Critical Limits

This section should clearly distinguish what was intentionally implemented from what remains outside the current PFE scope.

9.7 Chapter Conclusion

This chapter should show maturity, self-criticism, and a realistic understanding of the delivered platform.

Chapter 10

General Conclusion and Perspectives

10.1 General Conclusion

AdOptimizer CDP delivers a distributed and secured customer data lakehouse platform implemented within the broader AdOptimizer AI initiative at SMART AUTOMATION TECHNOLOGIES. The project demonstrates that customer-oriented data can be ingested, stored, processed, transformed, validated, exposed, orchestrated, observed, and protected within a coherent multi-node architecture.

10.2 Main Achievements

The project establishes a practical foundation that combines:

- distributed ingestion and storage,
- distributed processing,
- governed analytical exposure,
- orchestration,
- observability,
- and integrated security controls.

10.3 Future Perspectives

The platform can later support:

- Customer 360 consolidation,
- customer segmentation,
- churn prediction,
- lifetime value estimation,
- personalization services,
- and AI-driven marketing decision support.

10.4 Final Positioning

The final outcome should be defended as a strong data engineering and distributed systems contribution that prepares the path for future intelligent services without claiming to already be the final AI product.

Bibliography

Appendix A

Appendices

Appendix Organization

This appendix section is prepared to host the supporting material required for the final submission.

Recommended Appendix Contents

The final version can include:

- deployment commands,
- environment variables,
- configuration extracts,
- screenshots used as validation evidence,
- service access matrices,
- and operational troubleshooting notes.

Evidence Location

The corresponding raw support material should be organized primarily inside:

- 90_assets/
- 91_evidence/

- 99_submission_package/